Key Academic Articles for the Master's Thesis "Environment for Creating Music Using Programming"

Below are ten verifiable, peer-reviewed academic publications most relevant to a thesis on the design and C# implementation of a text-based DSL for music creation. Each entry lists the full bibliographic data, a verifiable source URL or DOI, a concise abstract summary, and a justification of relevance. After the list of ten, the **Top 5 most directly useful** papers for this specific thesis are highlighted.

The Ten Articles

1. McCartney, J. (2002). *Rethinking the Computer Music Language: SuperCollider*

- **Authors:** James McCartney
- **Year:** 2002
- **Source:** Computer Music Journal, Vol. 26, No. 4, pp. 61–68 (MIT Press) ACM Other conferences
- **DOI / URL:** https://doi.org/10.1162/014892602320991383
- **Abstract summary:** McCartney describes the design rationale of SuperCollider, an object-oriented language and runtime for real-time audio synthesis and algorithmic composition. The paper argues for a separation between a high-level dynamically typed composition language and an efficient unit-generator–based synthesis server, communicating via OSC. It explains how Smalltalk-style polymorphism, garbage collection, and an algorithmic instrument-graph generation model lead to a flexible computer-music environment. ResearchGate
- **Relevance:** A canonical reference for "language + audio backend" architecture — exactly the layering proposed in the thesis (DSL front end → internal representation → existing audio backend). Provides the architectural template that informs almost every subsequent music DSL.

2. Wang, G., Cook, P. R., & Salazar, S. (2015). *ChucK: A Strongly Timed Computer Music Language*

- **Authors:** Ge Wang, Perry R. Cook, Spencer Salazar
- **Year:** 2015
- **Source:** Computer Music Journal, Vol. 39, No. 4, pp. 10–29 (MIT Press)
- **DOI / URL:** https://doi.org/10.1162/COMJ_a_00324 (open PDF: https://ccrma.stanford.edu/~ge/publish/files/2015-cmj-chuck.pdf)
- **Abstract summary:** The paper formalises the design of ChucK, a programming language whose central innovation is the *strongly timed* programming model: time is a first-class

data type, and the keyword `now` advances logical time deterministically. The authors detail syntactic constructs for sample-accurate scheduling, concurrency via "shreds," and on-the-fly programming. Semantic Scholar

- **Relevance:** Directly addresses how to express rhythm and time semantics in a music DSL — one of the thesis's core requirements (describing rhythm and basic playback parameters in source code). Offers concrete operator/keyword designs that can inform the syntax of the new language. Semantic Scholar

3. McLean, A., & Wiggins, G. (2010). *Tidal – Pattern Language for the Live Coding of Music*

- **Authors:** Alex McLean, Geraint Wiggins
- **Year:** 2010
- **Source:** Proceedings of the 7th Sound and Music Computing Conference (SMC '10), Barcelona, pp. 331–334 ACM Digital Library
- **URL:** https://slab.org/publications/ (PDF: https://www.researchgate.net/publication/261134964)
- **Abstract summary:** Introduces Tidal, a pattern DSL embedded in Haskell that represents polyphonic patterns as time-varying functions, with combinators for hierarchical pattern construction. Patterns drive OSC messages to external synths; the language emphasises terse, expressive notation suitable for real-time editing. Academia.edu Academia.edu
- **Relevance:** The foundational paper on a domain-specific language for *describing musical patterns* — the exact problem the thesis addresses. Provides a concrete model for pattern combinators and rational-time semantics.

4. Aaron, S., Orchard, D., & Blackwell, A. F. (2014). *Temporal Semantics for a Live Coding Language*

- **Authors:** Samuel Aaron, Dominic Orchard, Alan F. Blackwell
- **Year:** 2014
- **Source:** Proceedings of the 2nd ACM SIGPLAN International Workshop on Functional Art, Music, Modelling and Design (FARM '14), pp. 37–47
- **DOI:** https://doi.org/10.1145/2633638.2633648 (open PDF: https://kar.kent.ac.uk/57489/1/sonicpi.pdf)
- **Abstract summary:** Presents the formal time-system and monadic denotational semantics underlying Sonic Pi v2.0. The paper explains why naive `sleep` semantics produce poorly timed concurrent music and introduces a "virtual time" model where `sleep` advances logical time without blocking sound scheduling, with a soundness proof relating axiomatic and denotational specifications.
- **Relevance:** Provides a rigorous specification of how time/rhythm can be encoded in the *semantics* of a music DSL — extremely valuable for the thesis chapter on syntax and

semantics design and for the compiler/interpreter that processes code into internal music events.

5. Aaron, S., & Blackwell, A. F. (2013). ***From Sonic Pi to Overtone: Creative Musical Experiences with Domain-Specific and Functional Languages***

- **Authors:** Samuel Aaron, Alan F. Blackwell
- **Year:** 2013
- **Source:** Proceedings of the 1st ACM SIGPLAN Workshop on Functional Art, Music, Modelling and Design (FARM '13), pp. 35–46
- **DOI:** https://doi.org/10.1145/2505341.2505346
- **Abstract summary:** An experience report describing two music DSLs: Sonic Pi (a Ruby-embedded teaching language) and Overtone (a Clojure-embedded live-coding environment). The authors document the architectural decisions — front-end DSL, mirrored state, OSC-based communication with SuperCollider — that translate user-level musical abstractions into low-level synthesis events.
- **Relevance:** A concrete case study of building a music DSL on top of an existing audio backend, exactly mirroring the thesis architecture. Shows pragmatic patterns for translating high-level musical syntax into scheduled audio events.

6. Sorensen, A., & Gardner, H. (2010). ***Programming with Time: Cyber-Physical Programming with Impromptu***

- **Authors:** Andrew Sorensen, Henry Gardner
- **Year:** 2010
- **Source:** OOPSLA '10 / ACM SIGPLAN Notices 45(10), pp. 822–834
- **DOI:** https://doi.org/10.1145/1869459.1869526
- **Abstract summary:** Argues for treating program execution as a temporally embedded cyber-physical activity and describes how the Impromptu (later Extempore) environment realises this view through temporally-recursive scheduling and just-in-time code rewriting. Discusses programming abstractions for managing latency, concurrency and continuous interaction with the physical world. QUT ePrints
- **Relevance:** Establishes the conceptual framework for designing time-aware language constructs in a music DSL — useful when defining how the thesis's compiler/interpreter handles scheduled playback events.

7. Hudak, P., Makucevich, T., Gadde, S., & Whong, B. (1996). ***Haskore Music Notation – An Algebra of Music***

- **Authors:** Paul Hudak, Tom Makucevich, Syam Gadde, Bo Whong

- **Year:** 1996
- **Source:** Journal of Functional Programming, Vol. 6, No. 3, pp. 465–483 (Cambridge University Press)
- **DOI:** https://doi.org/10.1017/S0956796800001805
- **Abstract summary:** Defines Haskore, an embedded DSL in Haskell for symbolic music description. Music is built from primitive notes/rests via sequential and parallel composition, transposition, and tempo scaling. The authors prove algebraic identities of these operators and define a "literal performance" semantics that maps abstract music to concrete event lists.
- **Relevance:** Provides a clean, formal model for an *internal music event representation* — directly applicable to the thesis's intermediate representation. Demonstrates how compositional operators (sequence, parallel, transpose) can be designed orthogonally and given precise semantics.

8. Aaron, S., Blackwell, A. F., Hoadley, R., & Regan, T. (2011). *A Principled Approach to Developing New Languages for Live Coding*

- **Authors:** Samuel Aaron, Alan F. Blackwell, Richard Hoadley, Tim Regan
- **Year:** 2011
- **Source:** Proceedings of the International Conference on New Interfaces for Musical Expression (NIME 2011), Oslo, pp. 381–386
- **URL:** https://www.nime.org/proc/aaron2011/ (Zenodo: https://zenodo.org/records/1177935)
- **Abstract summary:** Introduces the *Improcess* project and proposes a 3-tier architecture organised around a "Common Music Runtime" — a shared event-level substrate over which multiple front-end DSLs and hardware controllers can co-operate. The paper advocates principled separation of musical-process semantics from surface syntax.
- **Relevance:** Provides explicit design guidance for layering a custom music DSL over a shared internal event model and an existing backend — the precise architectural shape proposed in the thesis.

9. Collins, N., McLean, A., Rohrhuber, J., & Ward, A. (2003). *Live Coding in Laptop Performance*

- **Authors:** Nick Collins, Alex McLean, Julian Rohrhuber, Adrian Ward
- **Year:** 2003
- **Source:** Organised Sound, Vol. 8, No. 3, pp. 321–330 (Cambridge University Press)
- **DOI:** https://doi.org/10.1017/S135577180300030X
- **Abstract summary:** A foundational survey of live-coding practice in laptop music performance. The authors examine how programmers build and modify code in real time using languages such as SuperCollider with the Just-In-Time library and bespoke

Perl/REALbasic systems, articulating early principles of expressiveness, immediacy, and risk. Taylor & Francis Online

- **Relevance:** A primary citation for the analysis chapter on existing music programming languages, providing historical and practice-based context that frames Sonic Pi, TidalCycles, SuperCollider, and ChucK.

10. Fernández, J. D., & Vico, F. (2013). ***AI Methods in Algorithmic Composition: A Comprehensive Survey***

- **Authors:** José David Fernández, Francisco Vico
- **Year:** 2013
- **Source:** Journal of Artificial Intelligence Research, Vol. 48, pp. 513–582
- **DOI:** https://doi.org/10.1613/jair.3908 (open PDF: https://arxiv.org/abs/1402.0585)
- **Abstract summary:** A broad survey of computational techniques for algorithmic composition, covering grammars, Markov chains, constraint programming, evolutionary algorithms, neural networks, and rule-based symbolic systems. Discusses how compositional knowledge can be encoded in formal languages and tools.
- **Relevance:** Provides the wider intellectual context of "describing music as code," helps the thesis position its DSL within the algorithmic-composition tradition, and supplies reference material for the literature-review chapter.

Top 5 Most Directly Useful Papers for the Thesis

Selected on the basis of (i) DSL design for music, (ii) compiler/architecture detail of existing music languages, (iii) explicit treatment of pattern/rhythm/time semantics, and (iv) practical applicability to a C# desktop implementation that processes code into internal events for an external audio backend.

1. Aaron, Orchard & Blackwell (2014) — ***Temporal Semantics for a Live Coding Language.*** The single most directly applicable paper: it formalises the time-system and denotational semantics for a real, working music DSL (Sonic Pi). The thesis can re-use this framework when defining how its compiler maps source code to scheduled music events while preserving rhythmic correctness.

2. McLean & Wiggins (2010) — ***Tidal: Pattern Language for the Live Coding of Music.*** Defines the canonical model for representing musical patterns as functions of time and shows how pattern combinators yield expressive, concise notation. Directly informs the thesis's syntax for "musical patterns and rhythm."

3. Aaron & Blackwell (2013) — ***From Sonic Pi to Overtone.*** The closest architectural analogue to the thesis: a high-level DSL front end + mirrored state + OSC/event communication with an

external audio backend. Provides pragmatic implementation lessons that translate well to a C# Windows-Desktop setting.

4. Wang, Cook & Salazar (2015) — *ChucK: A Strongly Timed Computer Music Language.* The definitive description of "strongly timed" semantics, with concrete language constructs (`now`, the chuck operator `=>`, shreds). Essential reading for designing the syntax and semantics of time and concurrency in the new DSL. Semantic Scholar

5. McCartney (2002) — *Rethinking the Computer Music Language: SuperCollider.* The seminal blueprint for separating the high-level music language from the synthesis engine — the same separation the thesis adopts (custom DSL + existing audio backend). Provides core architectural justification and design patterns referenced by virtually all later work in this area.

Notes on Verification and Source Quality

All ten entries are peer-reviewed publications appearing in major venues (Computer Music Journal, JFP, JAIR, Organised Sound) or principal community conferences (ACM FARM, ACM OOPSLA/SIGPLAN, NIME, SMC, ICMC). DOIs and stable institutional URLs were cross-checked against multiple independent sources (ACM DL, MIT Press, Cambridge Core, NIME proceedings archive, university repositories at Kent, Sussex, Stanford CCRMA, and Yale). For McLean & Wiggins (2010), the paper appears in non-DOI conference proceedings (SMC 2010); the canonical citation is reproduced from the author's official publications page and is referenced verbatim in dozens of later peer-reviewed works. The Fernández & Vico survey is referenced both via its JAIR DOI and its open arXiv mirror to ensure access.

Two further well-known works that came up during research — Magnusson's "ixi lang: A SuperCollider Parasite for Live Coding" (ICMC 2011) and McLean's "Making Programming Languages to Dance to" (FARM 2014, doi:10.1145/2633638.2633647) — were strong candidates but were excluded from the final ten in favour of the more architecturally informative selections above; they remain useful supplementary reading should the thesis wish to broaden its survey of existing music DSLs. University of Brighton + 2